

Projet 1A : Atelier Git

Plateforme de travail : GitHub

Introduction

Le but de ce tutoriel est de vous initier à l'utilisation de l'outil **Git** sous la plateforme **GitHub**. Cet outil vous facilitera le travail collaboratif avec les membres de votre équipe afin de bien gérer vos projets.

1) Présentation de Git

Git est un logiciel de gestion de versions décentralisé. Ce type de programmes a plusieurs fonctions, notamment :

- Garder un historique des différentes versions des fichiers d'un projet ;
- Permettre le retour à une version ultérieure quelconque ;
- Garder un historique des modifications avec leur nature, leur date, leur auteur... ;
- Permettre un accès souple à ces fichiers, en local ou via un réseau ;
- Permettre à des utilisateurs distincts et souvent distants de travailler ensemble sur les mêmes fichiers.

Git est un système décentralisé. Toutes les machines ont accès à la base de code, pas besoin de passer par un serveur central.

Le modèle distribué présente plusieurs avantages :

- Moins de risques de perdre le code puisqu'il est accessible par plusieurs sources.
- On peut travailler plus rapidement et sans être connecté à Internet puisqu'il n'y a pas besoin de se connecter à un serveur central.

En plus des avantages du modèle distribué, Git a un autre atout : une grande communauté ! Cela facilite la collaboration et les échanges fructueux entre développeurs.

Avant de commencer cet atelier, vous devez tout d'abord installer l'outil Git sur vos

machines.

Pour windows :

Télécharger la dernière version depuis ce lien : <https://gitforwindows.org/>

Pour Ubuntu :

Taper la commande suivante dans le terminal linux: `sudo apt-get install git`

ou bien `sudo apt-get install git-core gitk`

Ensuite taper votre mot de passe

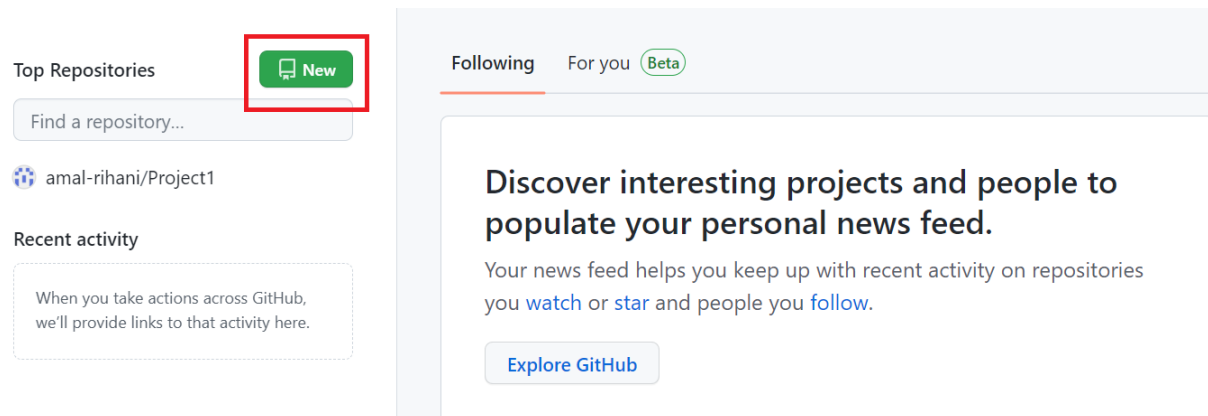
2) Création du compte

La première chose à faire est évidemment de créer un compte. Pour ceci :

1. Allez sur la page de GitHub: <https://github.com/>
2. Créer un compte.

3) Création d'un répertoire

N'oublions pas que notre but est de monitorer nos projets, donc nous allons commencer par créer un nouveau répertoire. Pour ce faire, cliquez sur « **New** » qui se trouve à la page d'accueil, ou bien, en haut à droite de la page, utilisez le menu déroulant «+» et cliquez sur « **New repository** ».



Saisissez les informations relatives à votre projet. Ci-dessous un exemple de création de projet:

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Ajouter le nom du projet

Owner *

 amal-rihani ▾

Repository name *

Nom_du_Projet ✓

Great repository names are short and memorable. Need inspiration? How about [symmetrical-broccoli?](#)

Description (optional)

Ajouter une description du projet

Description du projet

☐



Public

Anyone on the internet can see this repository. You choose who can commit.

☒



Private

You choose who can see and commit to this repository.

Choisir l'option "Private"

Initialize this repository with:

Skip this step if you're importing an existing repository.

☐ Add a README file

This is where you can write a long description for your project. [Learn more.](#)

Add .gitignore

Choose which files not to track from a list of templates. [Learn more.](#)

.gitignore template: None ▾

Choose a license

A license tells others what they can and can't do with your code. [Learn more.](#)

License: None ▾

 You are creating a private repository in your personal account.

Create repository

4) Personnalisation du projet

Une fois votre répertoire est créé, la page suivante s'affiche.

amal-rihani / **Nom_du_Projet** Private

Unwatch 1

Code Issues Pull requests Actions Projects Security Insights **Settings**

Configurer le projet

Quick setup — if you've done this kind of thing before

Set up in Desktop or HTTPS SSH **https://github.com/amal-rihani/Nom_du_Projet.git**

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# Nom_du_Projet" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/amal-rihani/Nom_du_Projet.git
git push -u origin main
```

Comme vous pouvez le constater, vous pourrez modifier les caractéristiques de votre projet à travers la page « **Settings** ». Pour le moment, nous allons nous contenter d'ajouter des membres dans le répertoire. Tous les membres doivent posséder des comptes sur GitHub et il suffira alors de saisir le « **username** » du membre et cliquer sur « **add collaborator** » comme illustré ci-dessous.

Search or jump to... Pull requests Issues Codespaces Marketplace Explore

amal-rihani / **Nom_du_Projet** Private

Unwatch 1 Fork 0 Star 0

Code Issues Pull requests Actions Projects Security Insights **Settings**

General

Access

Collaborators

Code and automation

Actions Webhooks Codespaces Pages

Repository name

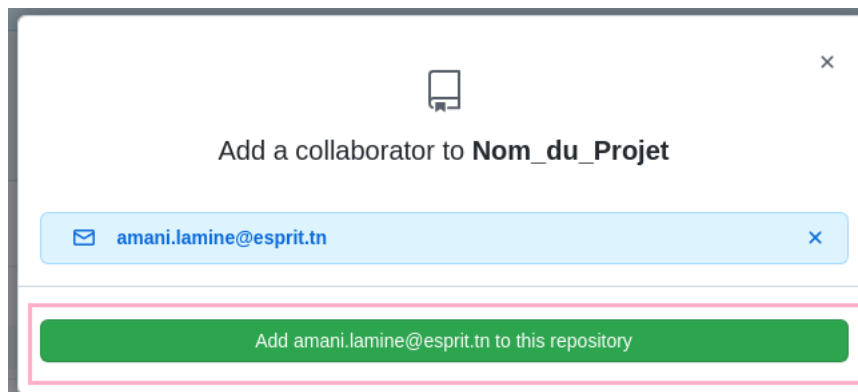
Nom_du_Projet Rename

☐ Template repository

Template repositories let users generate new repositories with the same directory structure and files. [Learn more.](#)

☒ Require contributors to sign off on web-based commits

Enabling this setting will require contributors to sign off on commits made through GitHub's web interface. Signing off is a way for contributors to affirm that their commit complies with the repository's terms, commonly the [Developer Certificate of Origin \(DCO\)](#). [Learn more about signing off on commits.](#)

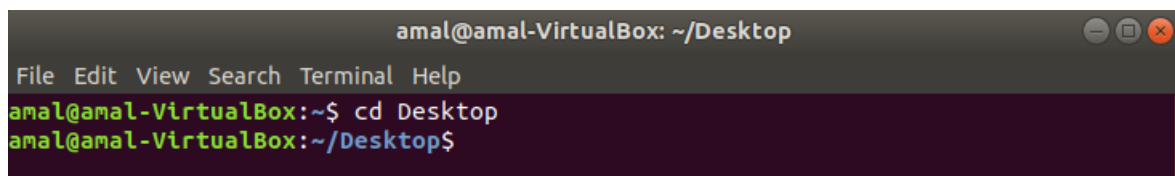


5) Création d'une copie de travail local

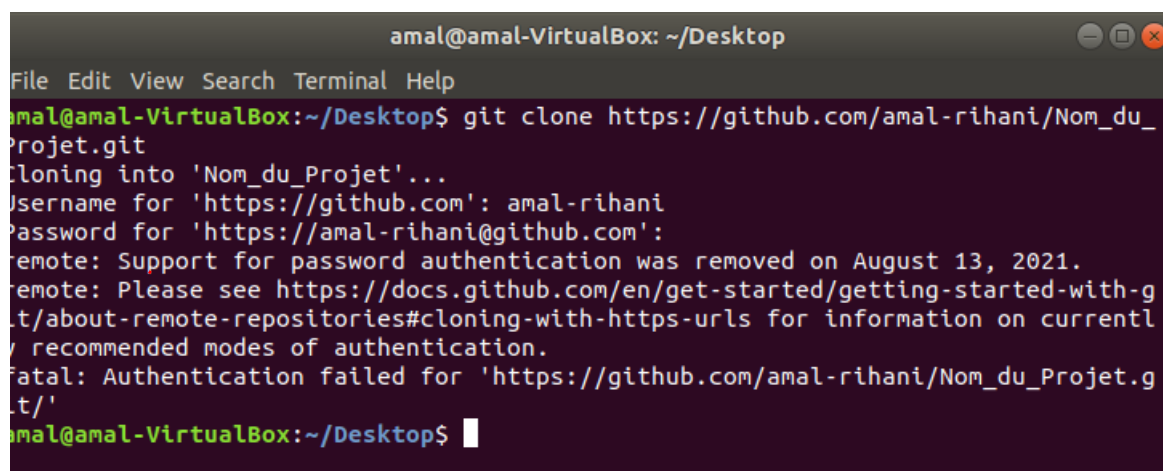
En ouvrant le **terminal d'Ubuntu**, vous devez définir votre **nom** ainsi que votre **mail** avec la commande suivante:

```
amal@amal-VirtualBox:~$ git config --global user.name "amal-rihani";
amal@amal-VirtualBox:~$ git config --global user.email "amal.rihani@esprit.tn";
```

Par la suite déplacez vous vers le bureau avec la commande « **cd desktop** » .

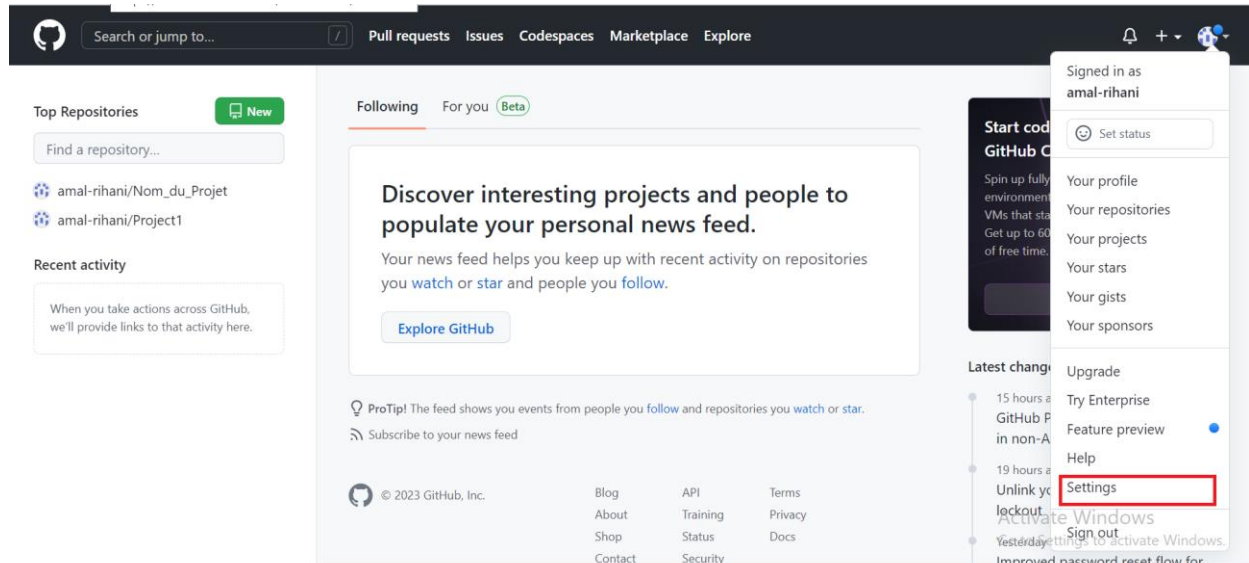


Pour récupérer le dossier « **Nom_du_Projet** », crée dans **GitHub**, sur votre machine, on aura besoin du lien du répertoire. Pour cela cliquez sur « **code** » puis copiez le lien de votre projet. Taper ensuite la commande suivante: « **git clone** » afin de télécharger le projet « **Nom_du_Projet** » sur votre PC.

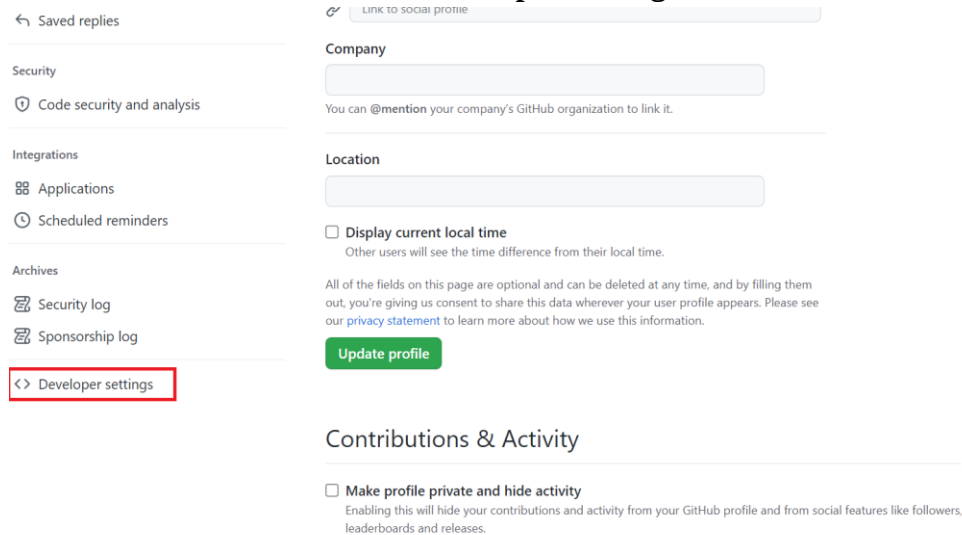


Cette erreur s'affichera. Pour résoudre ce problème, il faut **créer un Token** en suivant les étapes ci-dessous:

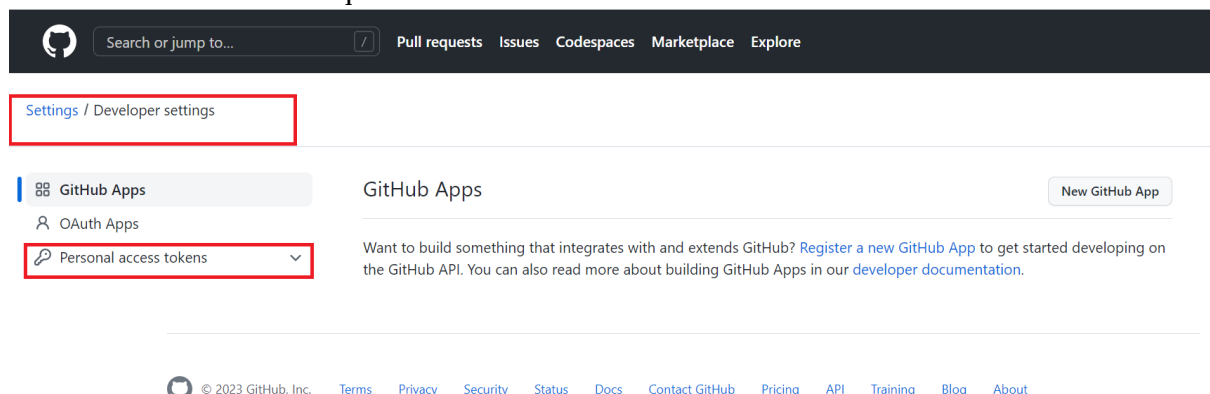
- Connectez-vous à votre compte GitHub
- Cliquez sur « **Settings** »



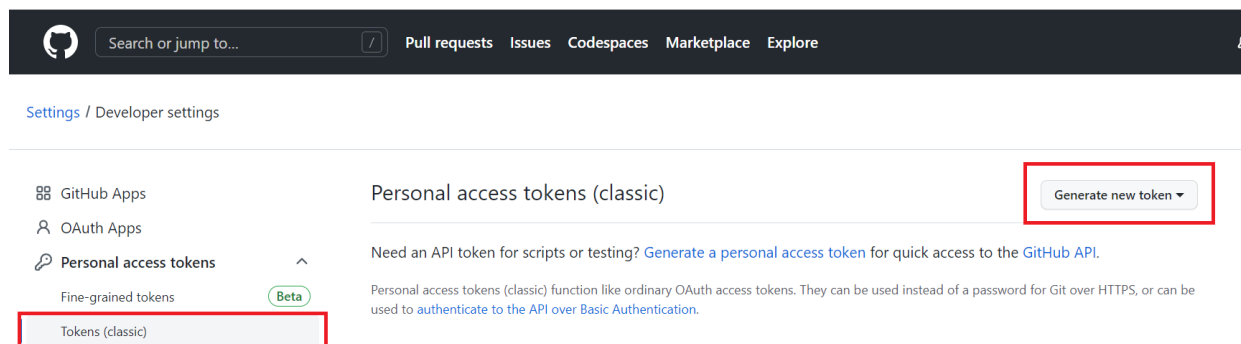
- Choisir « **Developer Settings** »



- Cliquez sur « **Personal access tokens** »



- « **Generate new token** »



GitHub logo Search or jump to... Pull requests Issues Codespaces Marketplace Explore

Settings / Developer settings

GitHub Apps OAuth Apps Personal access tokens ^

Fine-grained tokens (Beta)

Tokens (classic)

Personal access tokens (classic)

Generate new token ▼

Need an API token for scripts or testing? [Generate a personal access token](#) for quick access to the GitHub API.

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

- **Choisissez un nom**, sélectionnez la date d'expiration et cochez les champs d'accès suivants, puis générez le token.

- GitHub Apps
- OAuth Apps
- Personal access tokens
 - Fine-grained tokens
 - Tokens (classic)**

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note

New_token

What's this token for?

Expiration *

90 days

The token will expire on Tue, May 23 2023

Select scopes

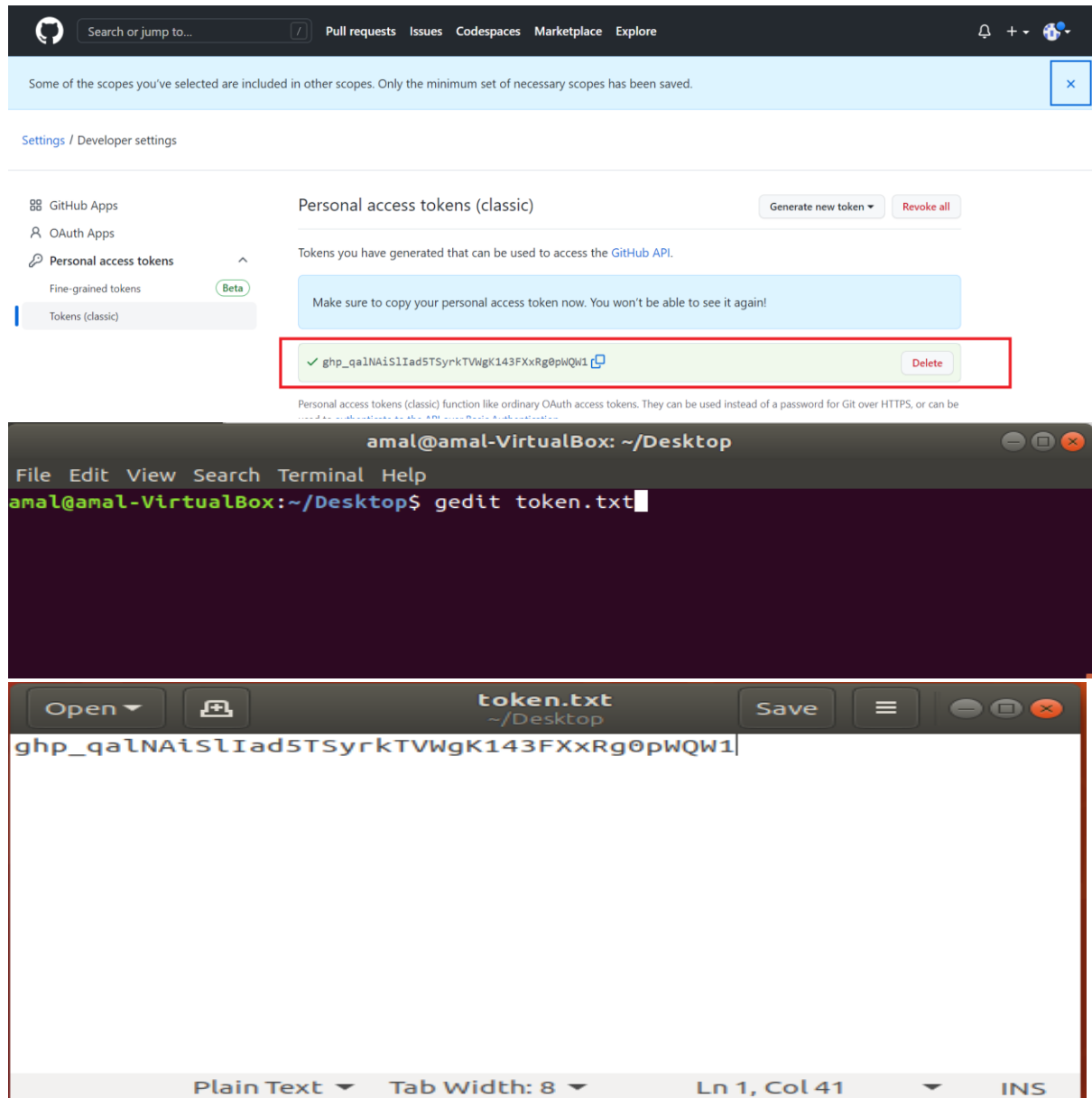
Scopes define the access for personal tokens. [Read more about OAuth scopes](#).

☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo:deployment	Access deployment status
☒ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☒ security_events	Read and write security events
☐ workflow	Update GitHub Action workflows
☐ write:packages	Upload packages to GitHub Package Registry
☐ read:packages	Download packages from GitHub Package Registry
☐ delete:packages	Delete packages from GitHub Package Registry
☐ admin:org	Full control of orgs and teams, read and write org projects
☐ write:org	Read and write org and team membership, read and write org projects
☐ read:org	Read org and team membership, read org projects
☐ manage_runners:org	Manage org runners and runner groups
☐ admin:public_key	Full control of user public keys
☐ write:public_key	Write user public keys
☐ read:public_key	Read user public keys
☒ admin:repo_hook	Full control of repository hooks
☒ write:repo_hook	Write repository hooks
☒ read:repo_hook	Read repository hooks
☐ admin:org_hook	Full control of organization hooks
☐ gist	Create gists
☐ notifications	Access notifications
☐ user	Update ALL user data
☐ read:user	Read ALL user profile data
☐ user:email	Access user email addresses (read-only)
☐ user:follow	Follow and unfollow users
☒ delete_repo	Delete repositories
☐ write:discussion	Read and write team discussions
☐ read:discussion	Read team discussions
☐ admin:enterprise	Full control of enterprises
☐ manage_runners:enterprise	Manage enterprise runners and runner groups
☐ manage_billing:enterprise	Read and write enterprise billing data
☐ read:enterprise	Read enterprise profile data
☐ audit_log	Full control of audit log
☐ read:audit_log	Read access of audit log
☐ codespace	Full control of codespaces
☐ codespace	Full control of codespaces
☐ codespace:secrets	Ability to create, read, update, and delete codespace secrets
☐ project	Full control of projects
☐ read:project	Read access of projects
☐ admin:gpg_key	Full control of public user GPG keys
☐ write:gpg_key	Write public user GPG keys
☐ read:gpg_key	Read public user GPG keys
☐ admin:ssh_signing_key	Full control of public user SSH signing keys
☐ write:ssh_signing_key	Write public user SSH signing keys
☐ read:ssh_signing_key	Read public user SSH signing keys

Generate token

Cancel

- Copiez votre Token d'accès et enregistrez-le dans un document texte.

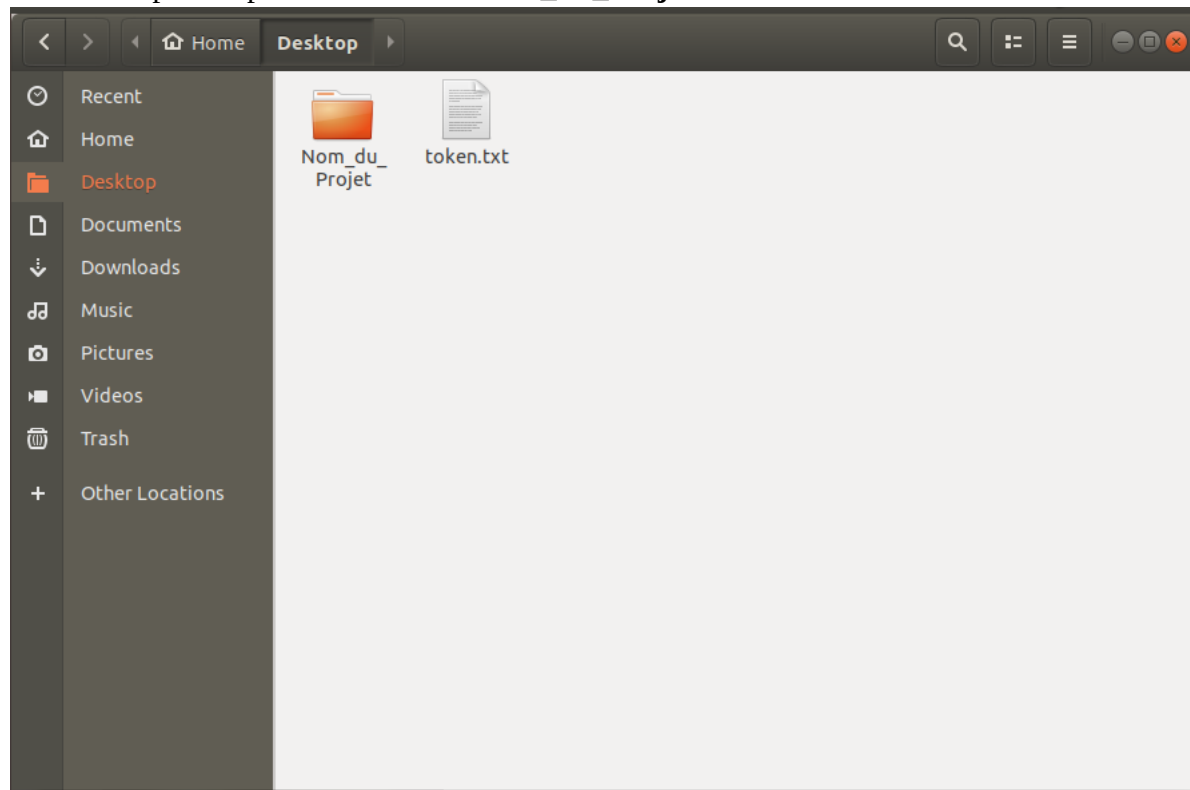


- Quand il vous demandera d'entrer le mot de passe Git, utilisez le Token d'accès sauvegardé.

Retaper de nouveau la commande « **git clone** »

```
amal@amal-VirtualBox:~/Desktop$ git clone https://github.com/amal-rihani/Nom_du_Projet.git
Cloning into 'Nom_du_Projet'...
Username for 'https://github.com': amal-rihani
Password for 'https://amal-rihani@github.com':
warning: You appear to have cloned an empty repository.
amal@amal-VirtualBox:~/Desktop$
```

Vous avez pu récupérer le dossier « **Nom_du_Projet** » sur votre PC.



6) Ajout des fichiers dans l'espace de travail collaboratif

Maintenant vous pouvez créer vos fichiers (code C, images, documents..) et les ajouter sous vos répertoires.

Déplacez-vous dans le dossier et créez, par exemple, un fichier **main.c** à l'aide de la commande « **gedit main.c** »

```
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ gedit main.c
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$
```

Pousser mettre à jour le contenu de votre copie locale :

1. Ajouter votre fichier à l'index avec la commande « **git add** »

```
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git add main.c
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$
```

2. Faire un commit qui décrit la mise à jour de votre fichier avec la commande « **git commit** »

```
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git commit -m "ajout du fichier main.c"
[master (root-commit) c552f7b] ajout du fichier main.c
1 file changed, 2 insertions(+)
create mode 100644 main.c
```

3. Poussez! Envoyez ces modifications sur votre répertoire créé sous GitHub en utilisant la commande **git push**: (vous devez fournir votre **username** et le **Token**)

```
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git push
Username for 'https://github.com': amal-rihani
Password for 'https://amal-rihani@github.com':
Counting objects: 3, done.
Writing objects: 100% (3/3), 228 bytes | 228.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0)
To https://github.com/amal-rihani/Nom_du_Projet.git
* [new branch]      master -> master
```

The screenshot shows the GitHub interface for a repository named 'Nom_du_Projet' by user 'amal-rihani'. The repository is marked as 'Private'. The top navigation bar includes links for 'Pull requests', 'Issues', 'Codespaces', 'Marketplace', and 'Explore'. Below the repository name, there are tabs for 'Code', 'Issues', 'Pull requests', 'Actions', 'Projects', 'Security', 'Insights', and 'Settings'. The 'Code' tab is selected. The main content area shows the repository's commit history. A red box highlights the most recent commit, which is titled 'ajout du fichier main.c' and has the commit hash 'c552f7b'. Below the commit title, there is a table showing the files changed in this commit: 'main.c' with the description 'ajout du fichier main.c'. To the right of the commit information, there are buttons for 'Go to file', 'Add file', and 'Code'. At the bottom of the commit list, there is a prompt to 'Add a README'.

7) Récupérer les modifications

Pour récupérer les modifications, il suffit d'utiliser la commande suivante: « **Git pull** ».

```
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git pull origin master
Username for 'https://github.com': amal-rihani
Password for 'https://amal-rihani@github.com':
From https://github.com/amal-rihani/Nom_du_Projet
* branch      master      -> FETCH_HEAD
Already up to date.
```

8) Historique avec Git log

Pour afficher les commits, vous pouvez lancer la commande: « **git log** »

```
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git log
commit c552f7b311700b85ae7463a25030082b88128f50 (HEAD -> master, origin/master)
Author: amal-rihani <amal.rihani@esprit.tn>
Date:   Fri Mar 10 16:56:49 2023 +0100

    ajout du fichier main.c
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$
```

9) Supprimer un fichier du répertoire de travail et de l'index

Pour supprimer le fichier main.c, utiliser la commande « **git rm main.c** »

```
amal@amal-VirtualBox: ~/Desktop/Nom_du_Projet
File Edit View Search Terminal Help
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git rm main.c
rm 'main.c'
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$
```

Pour sauvegarder la suppression faire un commit et un push, en utilisant les commandes suivantes: « **git commit -m** » et « **git push** »

```
amal@amal-VirtualBox: ~/Desktop/Nom_du_Projet
File Edit View Search Terminal Help
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git commit -m "supprimer un fichier"
[master 47a3ea0] supprimer un fichier
1 file changed, 2 deletions(-)
delete mode 100644 main.c
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$
```

```
amal@amal-VirtualBox: ~/Desktop/Nom_du_Projet
File Edit View Search Terminal Help
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git push
Username for 'https://github.com': amal-rihani
Password for 'https://amal-rihani@github.com':
Counting objects: 2, done.
Compressing objects: 100% (1/1), done.
Writing objects: 100% (2/2), 237 bytes | 237.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0)
To https://github.com/amal-rihani/Nom_du_Projet.git
   ce39d2f..47a3ea0  master -> master
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$
```

10) Les branches

Le principal atout de Git est son système de branches. Sous Git, la branche principale est appelée la **branche main, ou master**. La branche principale (main ou master) portera l'intégralité des modifications effectuées. Le but n'est donc pas de réaliser les modifications directement sur cette branche, mais de les réaliser sur d'autres branches, et après divers tests, de les intégrer sur la branche principale.

En d'autre termes, chaque membre va créer sa propre branche, envoyer le travail sur cette dernière et comme **dernière étape**, l'intégrer sur la branche principale.

Tout d'abord, nous allons commencer par lister les branches existantes en utilisant « **git branch** »

Dans un premier temps, vous n'aurez que la branche master:

```
amal@amal-VirtualBox: ~/Desktop/Nom_du_Projet
File Edit View Search Terminal Help
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git branch
* master
```

Et c'est normal. L'étoile signifie que c'est la branche sur laquelle vous vous situez, et que c'est sur celle-ci qu'actuellement vous réalisez vos modifications.

Nous avons donc notre branche master, et nous souhaitons maintenant créer une nouvelle branche. Pour cela, on tape « **git branch dev** » :

```
amal@amal-VirtualBox: ~/Desktop/Nom_du_Projet
File Edit View Search Terminal Help
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git branch dev
```

Cette commande va créer la branche **dev** en local.

Vous pouvez de nouveau taper la ligne de commande :

```
amal@amal-VirtualBox: ~/Desktop/Nom_du_Projet
File Edit View Search Terminal Help
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git branch
dev
* master
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$
```

Vous pouvez maintenant voir votre branche principale (master) et votre branche dev. Comme vous pouvez le voir, la petite étoile est toujours sur la branche master. Nous avons créé la branche dev, mais nous n'avons pas encore basculé sur celle-ci.

Pour basculer de branche, nous allons utiliser la commande « **git checkout dev** »:

```
amal@amal-VirtualBox: ~/Desktop/Nom_du_Projet
File Edit View Search Terminal Help
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git checkout dev
Switched to branch 'dev'
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$
```

Vous aurez donc :

```
amal@amal-VirtualBox: ~/Desktop/Nom_du_Projet
File Edit View Search Terminal Help
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git branch
* dev
  master
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$
```

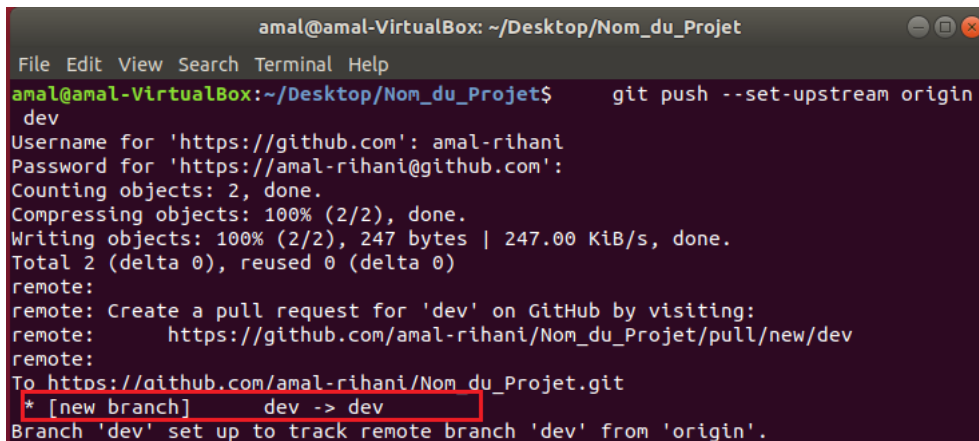
Maintenant, nous allons ajouter et faire un commit du fichier main.c sur la branche dev, nous allons procéder comme suit:

```
amal@amal-VirtualBox: ~/Desktop/Nom_du_Projet
File Edit View Search Terminal Help
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git branch
* dev
  master
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git add main.c
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git commit -m "ajout main.c cote d
ev"
[dev 5351348] ajout main.c cote dev
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 main.c
```

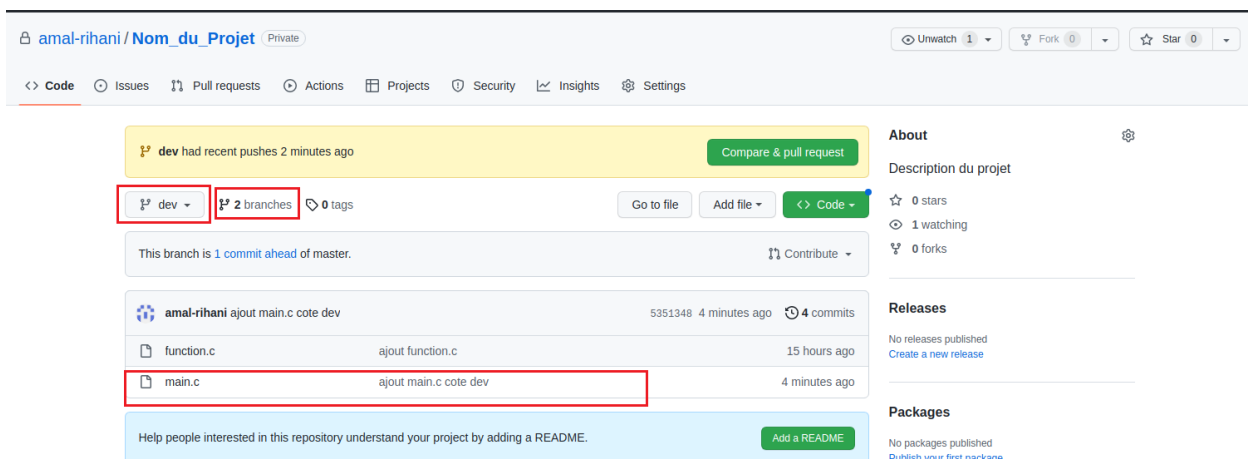
Votre fichier est maintenant enregistré sur la branche dev avec la description “ajout main.c cote dev”.

Avec la commande **git commit**, nous avons enregistré des modifications en local uniquement. Il ne vous reste plus qu'à envoyer les modifications réalisées sur le GitHub grâce à la commande « **git push --set-upstream origin dev** » sur la branche

dev comme le montre la figure ci-après:



```
amal@amal-VirtualBox: ~/Desktop/Nom_du_Projet
File Edit View Search Terminal Help
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git push --set-upstream origin dev
Username for 'https://github.com': amal-rihani
Password for 'https://amal-rihani@github.com':
Counting objects: 2, done.
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 247 bytes | 247.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0)
remote:
remote: Create a pull request for 'dev' on GitHub by visiting:
remote:   https://github.com/amal-rihani/Nom_du_Projet/pull/new/dev
remote:
To https://github.com/amal-rihani/Nom_du_Projet.git
* [new branch]      dev -> dev
Branch 'dev' set up to track remote branch 'dev' from 'origin'.
```

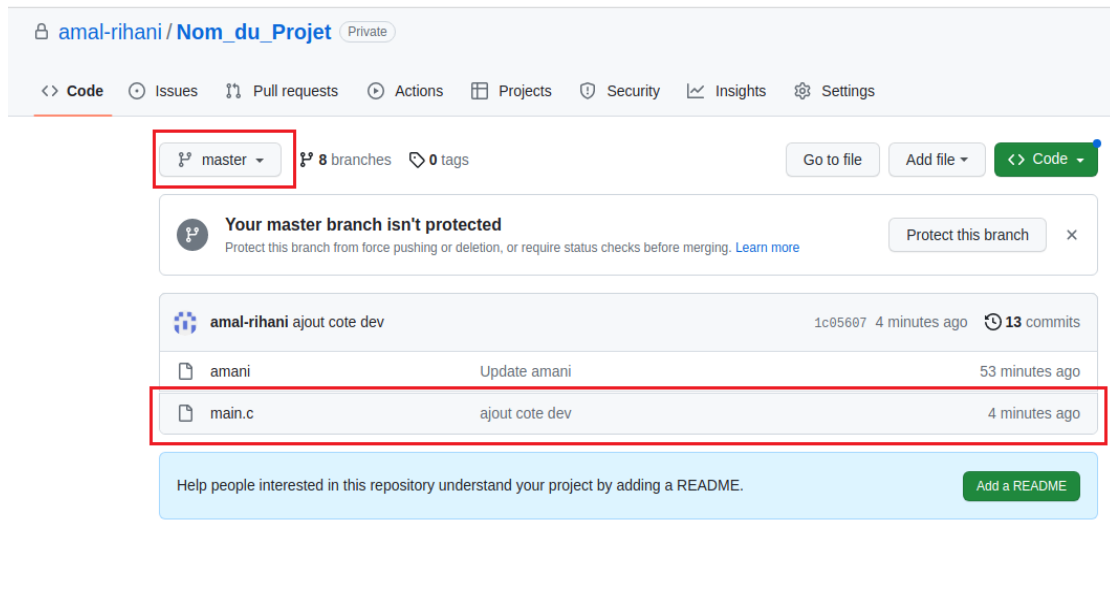


Intégration

À présent, il faut intégrer le travail réalisé dans la branche **dev** à la branche principale **master**. Pour cela, vous devez utiliser la commande « **git merge dev** » puis la commande « **git push** » .

Cette commande doit s'utiliser à partir de la branche dans laquelle nous voulons apporter les modifications. Dans notre cas, la commande s'effectuera donc dans la branche principale. Pour y retourner, utilisez la commande « **git checkout master** » comme le montre la figure ci-dessous:

```
amal@amal-VirtualBox: ~/Desktop/Nom_du_Projet
File Edit View Search Terminal Help
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git checkout master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git merge dev
Updating fd84dia..1c05607
Fast-forward
 main.c | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 main.c
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git push
Username for 'https://github.com': amal-rihani
Password for 'https://amal-rihani@github.com':
Total 0 (delta 0), reused 0 (delta 0)
To https://github.com/amal-rihani/Nom_du_Projet.git
 fd84dia..1c05607 master -> master
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$
```



Pour pouvoir afficher l'historique des différents commits réalisées sur les différentes branches, nous pouvez utiliser la commande « **git log** ».

```
amal@amal-VirtualBox: ~/Desktop/Nom_du_Projet
File Edit View Search Terminal Help
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git log
commit 53513483c19f02da5970be20db2e4f6d87cc3b18 (HEAD -> master, origin/dev dev)
Author: amal-rihani <amal.rihani@esprit.tn>
Date: Sat Mar 11 08:59:15 2023 +0100
    ajout main.c cote dev

commit 47a3ea04f8634259bc4d108ecb92929d2f056e80 (origin/master)
Author: amal-rihani <amal.rihani@esprit.tn>
Date: Fri Mar 10 18:56:53 2023 +0100
    supprimer un fichier

commit ce39d2f6480540a3d7f5eb04b4841c4dc7c5b79e
Author: amal-rihani <amal.rihani@esprit.tn>
Date: Fri Mar 10 17:42:32 2023 +0100
    ajout function.c

commit c552f7b311700b85ae7463a25030082b88128f50
Author: amal-rihani <amal.rihani@esprit.tn>
Date: Fri Mar 10 16:56:49 2023 +0100
```

Cette commande, vous liste des modifications apportées sur la branche **dev** (l'ajout du fichier main.c) et celles réalisées sur la branche **master** (suppression d'un fichier et l'ajout du fichier fonction.c).

11) Se positionner sur un commit bien précis

Lorsque vous effectuez une série de commits sur un projet, il peut vous arriver de vouloir remonter dans le temps à la recherche d'erreurs éventuelles par exemple. Pour vous positionner sur un commit donné de votre historique, il vous suffit d'utiliser la commande git checkout: git checkout SHA (le SHA est l'identifiant unique du Commit: à copier coller du résultat de la commande git log)

```
amal@amal-VirtualBox: ~/Desktop/Nom_du_Projet
File Edit View Search Terminal Help
amal@amal-VirtualBox:~/Desktop/Nom_du_Projet$ git checkout 47a3ea04f8634259bc4d108ecb92929d2f056e80
HEAD is now at 47a3ea0 supprimer un fichier
```

Conclusion

Ce tutoriel est une initiation aux bonnes pratiques du travail collaboratif entre les membres d'une équipe à l'aide de l'outil Git.